

Caio Xavier da Silva

**Otimização de Regras de Análise Estática
baseada em LLMs: Um Framework Especializado
em Vulnerabilidades Críticas da Linguagem C**

São Paulo

2026

Caio Xavier da Silva

Otimização de Regras de Análise Estática baseada em LLMs: Um Framework Especializado em Vulnerabilidades Críticas da Linguagem C

Proposta de Trabalho de Conclusão de Curso apresentado ao SENAC Santo Amaro como requisito parcial para aprovação na disciplina de TCC1 do Bacharelado em Ciência da Computação.

SENAC – Serviço Nacional de Aprendizagem Comercial
Unidade Santo Amaro
Bacharelado em Ciência da Computação

Orientador: Leonardo Massayuki Takuno
Coorientador: Thyago Conchado Quintas

São Paulo
2026

Resumo

Este documento apresenta a proposta inicial do Trabalho de Conclusão de Curso (TCC1) desenvolvido no SENAC Santo Amaro. O objetivo deste trabalho é desenvolver e avaliar um *framework* iterativo baseado em LLMs capaz de sintetizar automaticamente regras de análise estática para a ferramenta SemGrep, como foco em vulnerabilidades críticas da linguagem C/C++. A justificativa baseia-se na necessidade de reduzir custos computacionais na inferência direta por IAs e mitigar a dependência de engenharia manual e rígida de regras.

Palavras-chave: análise estática. semgrep. LLM. regras. C. C++.

Sumário

1	INTRODUÇÃO	5
1.1	Contexto	6
1.2	Justificativa	8
1.3	Hipótese	10
1.4	Objetivo	10
1.4.1	Objetivo Geral	10
1.4.2	Objetivos específicos	10
2	REVISÃO BIBLIOGRÁFICA	12
2.1	Metodologia da pesquisa bibliográfica	12
2.2	Fundamentos	14
2.2.1	Análise Estática Tradicional	14
2.2.1.1	Árvore Sintática Abstrata	14
2.2.1.2	Grafos de Fluxo de Controle	15
2.2.1.3	Análise de Fluxo de Dados	16
2.2.1.4	Sensibilidade de Fluxo	17
2.2.1.5	Limitações da Análise Estática	18
2.2.2	Vulnerabilidades de Linguagem	18
2.2.2.1	<i>Common Weakness Enumeration</i>	19
2.2.2.2	<i>Common Vulnerabilities and Exposures</i>	19
2.2.3	Arquitetura Transformers	19
2.2.3.1	Linguagem Natural	19
2.2.3.2	Modelos de Linguagens de Grande Escala	19
2.2.4	Métricas de Avaliação	20
2.3	Referencial teórico	20
2.3.1	Análise estática baseada em regras	20
2.3.1.1	Árvore Sintática Abstrata	22
2.3.2	Aplicação de LLMs em análise estática	22
2.3.3	Abstração de Regras	22
2.4	Trabalhos relacionados	22
2.4.1	Trabalho 1: <i>KNighter: Transforming Static Analysis with LLM-Synthesized Checkers</i> (YANG et al., 2025)	22
2.4.2	Trabalho 2: <i>Generation, Migration and Optimization of Cross-Language Static-Analysis Rules Based on Large Language Models</i> (HOU et al., 2025)	25

2.4.3	Trabalho 3: <i>LLM-based Vulnerability Detection at Project Scale: An Empirical Study</i> (LI et al., 2026)	27
2.4.4	Síntese comparativa	30
3	DESENVOLVIMENTO	32
3.1	Solução proposta	32
3.1.1	Arquitetura do trabalho	32
3.2	Cronograma de trabalho	34
	REFERÊNCIAS BIBLIOGRÁFICAS	35

1 Introdução

Ferramentas de análise estática são amplamente empregadas no desenvolvimento de *software* e garantia de código (HOU et al., 2025). Por essa abordagem não necessitar de compilação e execução prévia do sistema, ela é considerada um método rápido e econômico para a identificação de problemas em seus estágios iniciais. Segundo Jiang et al. (2025), as ferramentas de análise estática emergiram como componentes indispensáveis nos fluxos de trabalho modernos, fornecendo mecanismos automatizados para detecção de defeitos, vulnerabilidades de segurança e violações de padrões de codificação.

A análise estática possui diversas vertentes. Um dos problemas principais dos métodos tradicionais é a diversidade de abordagens necessárias ao lidar com diferentes projetos. Uma das estruturas mais conhecidas é a Árvore de Sintaxe Abstrata, em inglês *Abstract Syntax Tree* (AST), uma representação intermediária que modela a estrutura hierárquica do código-fonte. ASTs fornecem uma representação de programa geralmente utilizada para análises insensíveis ao fluxo (*flow-insensitive*), onde a ordem sequencial das instruções não é estritamente necessária (MØLLER; SCHWARTZBACH, 2024).

Complementar a isso, utilizam-se os grafos de fluxo de controle, em inglês *Control Flow Graphs* (CFG). Enquanto a AST foca na sintaxe, o CFG é construído para modelar os possíveis caminhos de execução do programa, sendo essencial para análises sensíveis ao fluxo (*flow-sensitive*). CFGs são muitas vezes utilizados para a análise clássica de fluxo de dados, de modo que para cada nó do grafo, se atribui uma restrição que relaciona o valor do nó com seus vizinhos, sejam eles predecessores ou sucessores, para propagar informações ao longo desses nós até atingir um ponto fixo de saída, que a partir destes resultados matemáticos é possível chegar em uma conclusão sobre o programa (MØLLER; SCHWARTZBACH, 2024).

Uma vez que as vulnerabilidades são detectadas, o próximo desafio no ciclo de desenvolvimento é a sua correção. Como uma evolução natural à simples detecção de falhas, o campo de Reparo de Programa Automatizado, em inglês *Automated Program Repair* (APR), tem ganhado destaque na engenharia de *software*. Enquanto a análise estática tradicional se limita a apontar as violações, o APR propõe-se a dar um passo além: gerar automaticamente soluções e aplicar correções de código (*patches*) sem intervenção humana, visando reduzir os custos de depuração (TANG et al., 2021).

Contudo, apesar de sua utilidade, tanto as ferramentas clássicas de detecção quanto os métodos tradicionais de correção enfrentam graves desafios estruturais. Conforme apontado por Hou et al. (2025) e Tang et al. (2021), a eficácia destas abordagens é frequentemente limitada por sua dependência de lógica rígida, o que compromete severamente a sua extensibilidade. A Tabela 1 sintetiza as principais limitações destas abordagens clássicas apontadas na literatura.

Tabela 1 – Limitações da análise estática tradicional

Limitação	Descrição
Escalabilidade e Diversidade de <i>Bugs</i>	A dependência de verificações pré-definidas ou modelagens formais restringe as ferramentas a um subconjunto limitado de padrões de <i>bugs</i> . Isso prejudica a escalabilidade e a capacidade de detectar vulnerabilidades imprevistas ou casos extremos (<i>corner-cases</i>) (YANG et al., 2025).
Esforço Humano	A criação de regras personalizadas exige um alto nível de especialização técnica. A complexidade sintática torna a tarefa inacessível para desenvolvedores comuns, sendo inviável mesmo com treinamento de curto prazo para realizar uma regra eficiente (HOU et al., 2025). Pesquisas revelam que menos de 5% das regras de análise estática utilizadas na indústria são criadas por usuários comuns (HOU et al., 2025).
Falta de Extensibilidade Multilinguagem	A maioria das regras é desenvolvida com lógica rigidamente codificada (<i>hard-coded</i>) para uma linguagem de programação específica. A necessidade de construir e manter <i>parsers</i> específicos para cada nova linguagem exige alta especialização técnica, o que inviabiliza a portabilidade e gera altos custos na adaptação de analisadores para ecossistemas políglotas (HOU et al., 2025).
Falsos Positivos	Devido à complexidade dos sistemas de <i>software</i> e à riqueza de gramáticas de programação, é inviável que regras manuais prevejam todas as variações estruturais. Essa generalização falha resulta em uma abundância de alarmes imprecisos no mundo real (JIANG et al., 2025).

Fonte: Elaborado pelo autor (2026)

1.1 Contexto

Nos últimos anos, muitas técnicas foram desenvolvidas para minimizar alarmes falsos em analisadores estáticos (JIANG et al., 2025). Recentemente, Modelos de Linguagem de Grande Porte (LLMs) e sua ampla aplicação mudaram profundamente o domínio de análise estática, a maioria dessas técnicas empregando LLMs para compreender o código-fonte diretamente ou os dados das próprias ferramentas de análise estática, para determinar a presença de *bugs*, como sua localização e tipo (HOU et al., 2025).

Modelos de Linguagem de Larga Escala possuem o conhecimento prévio adquirido durante seu pré-treinamento em vastos repositórios de código aberto. Durante a inferência, essas capacidades podem ser direcionadas via engenharia de *prompt* (como o fornecimento de históricos de *commits* de correção no contexto) para extrair padrões sintáticos e semânticos de *bugs* específicos do sistema (TANG et al., 2021). A capacidade de analisar vários conteúdos sugere que LLMs são capazes de se adaptar a novos tipos de falhas sem a necessidade de criação explícita de regras (YANG et al., 2025).

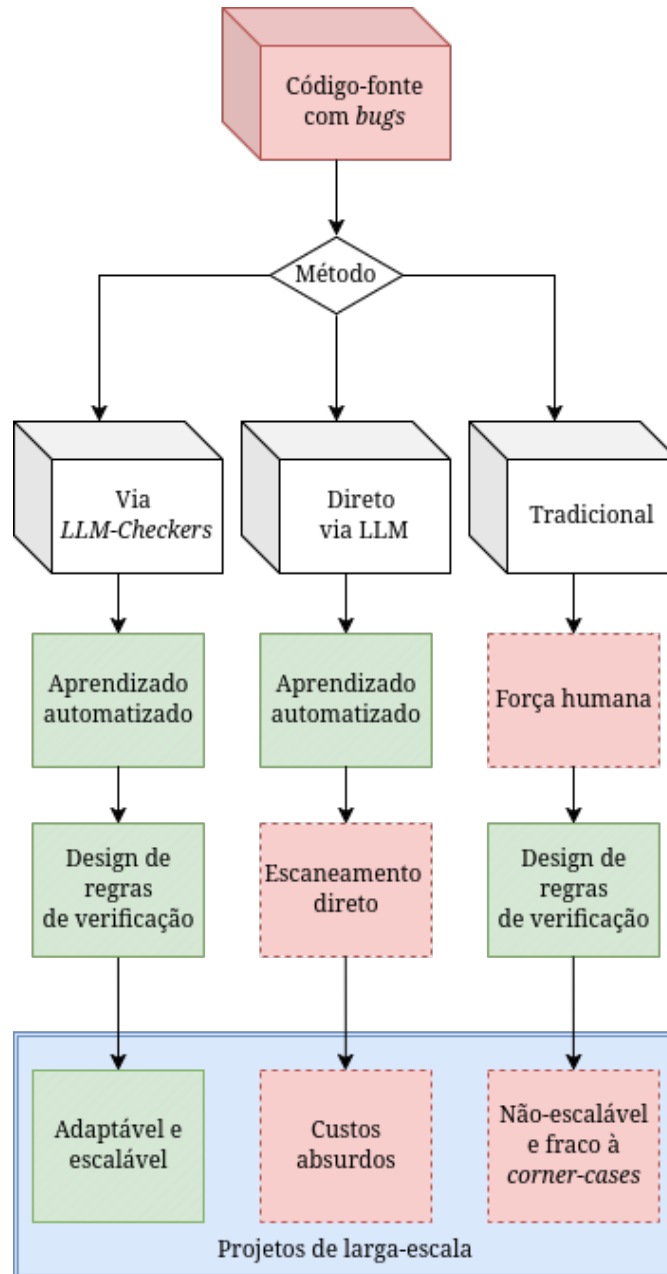


Figura 1 – Motivação para *checkers* feitos por llms.

Fonte: Adaptado pelo autor para melhor compreensão (YANG et al., 2025)

Contudo, as estratégias de pós-processamento dependem da inferência contínua das LLMs durante a inspeção de código (HOU et al., 2025). Apesar das janelas de contexto atuais atingirem a ordem de milhões de *tokens*, restrições práticas persistem. A submissão integral do código-fonte, aliada às iterações exigidas a cada modificação, não apenas acarreta custos computacionais proibitivos, como também exacerba o risco de alucinações, gerando inferências plausíveis, porém tecnicamente incorretas. Especialmente com sistemas de grande escala complexos (como exemplo, o *kernel* do Linux, com mais de 50 milhões de linhas de código) (YANG et al., 2025).

A figura 1 introduz motivações para a necessidade de métodos mais eficientes e,

principalmente, especializados, tal que em projetos grandes, a análise estática possui dois desafios: criar regras para diversos padrões de bugs e gerenciamento de bases de códigos imensas (YANG et al., 2025). Em contrapartida, a inspeção direta de código por LLMs esbarra em restrições de janela de contexto e implica em altos custos computacionais.

Considerando esses dados, o analisador estático ideal deveria: detectar uma diversidade de falhas, incluindo *corner-cases* e semânticas específicas de sistemas, e eficientemente processar milhões de linhas de código (YANG et al., 2025). Porém, dos métodos existentes, geralmente sacrificam um desses objetivos (YANG et al., 2025).

O método apresentado por Yang et al. (2025), KNighter, é um analisador estático sintetizado utilizando LLMs, que ao invés de métodos diretos de utilização de LLMs, é realizado um treinamento de padrões de *bugs* a partir de histórico de *patches* que a partir disso é dedicado para um *checker* especializado. A partir desse paradigma, Yang et al. (2025) sobrepassa os problemas de limites de contextos e custos absurdos de analisar bases de códigos de milhões de linhas com diversos bugs.

1.2 Justificativa

Apesar das conquistas e avanços que Yang et al. (2025) realizou, Hou et al. (2025) mencionam uma barreira fundamental relacionada à diversidade de linguagens. Até onde sabemos, o KNighter é um método direcionado à projetos C/C++ utilizando um *framework Clang Static Analysis*. A sua expansão para outros ecossistemas exigiria extensos ajustes nesse paradigma (HOU et al., 2025).

A partir dessa limitação, Hou et al. (2025) introduziu a metodologia de *prompting* para abordagem de diferentes linguagens, utilizando poucos exemplos (*few-shot learning*) como aprendizado. Os autores citam a ferramenta *Semgrep*, uma ferramenta utilizada para montagem de regras personalizadas para análise estática, como na Figura 2, tal que a utilizam para o *ruleset* que geram via LLMs.

A proposta sendo um *framework* baseado em LLMs para regras de análise estática para lidar com as limitações de escrever regras lidando com todos os possíveis casos (*corner-cases*) (HOU et al., 2025). Integrando o domínio de conhecimento adquirido de LLMs durante pré-treinamento não-supervisionado, integrando com as descrições de regras otimizadas para guiar a geração delas pelas LLMs.

Com essa metodologia, Hou et al. (2025) afirma, utilizando a ferramenta *SemGrep* e seu *ruleset* especializado, que seu método atingiu 81,99% de validação gramatical de suas regras pela ferramenta e 74,73% sendo regras funcionais e válidas. Apesar de seu *framework* possuir um foco em diversidade de linguagens, Hou et al. (2025) superou a quantidade de regras válidas de KNighter (YANG et al., 2025), de 37 regras para 41 regras, analisando tipos de *bugs* como *Null Pointer Dereference* (CWE-476), *Memory Leak* (CWE-401), *Use After Free* (CWE-416), *Double Free* (CWE-415), *Use of Uninitialized Variable* (CWE-457),

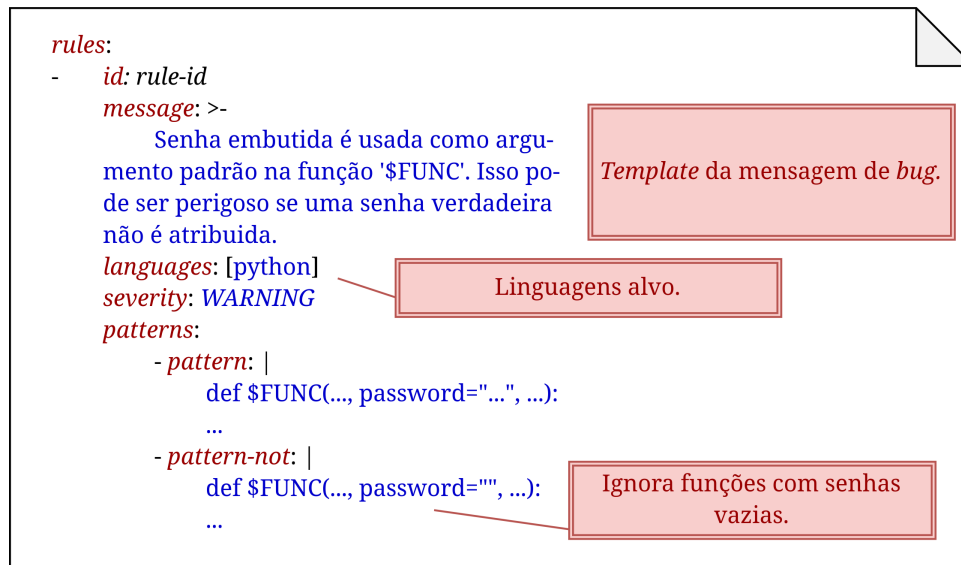


Figura 2 – Exemplo de regra criada para detectar uma senha embutida no argumento da função em Python.

Fonte: Adaptado para português com finalidade de melhorar compreensão (HOU et al., 2025).

Buffer Overflow (CWE-120), *Out-of-Bounds* (CWE-119), *Integer Overflow* (CWE-190), *Race Condition* (CWE-362) e *API Misuse* (CWE-227), utilizando de base e foco o projeto *open-source* Linux.

Além do KNighter, foi comparado sua performance para identificação de bugs entre *rulesets* oficiais de SemGrep e ferramentas de análise estática tradicionais como Smatch e CPPCheck. Onde entre todos os avisos e erros que as ferramentas obtiveram, nenhuma identificou defeitos tal que o método de Hou et al. (2025) identificou.

Outro trabalho que desafia esse nível de eficiência é Li et al. (2026). Seu *framework* analisa 222 vulnerabilidades conhecidas de 8 tipos de CWEs em C/C++ e Java, além de 24 projetos *open-source* ativos. Apesar de sua análise de identificação de falsos positivos serem realizadas manualmente, somando 150 horas de trabalho humano, Li et al. (2026) conseguiu concluir as causas desses falsos positivos e sua eficácia em detecção.

Sua primeira conclusão é que métodos de análise estática baseado em LLMs exibem uma cobertura limitada de vulnerabilidades do mundo real (LI et al., 2026). Os autores explicam que os métodos utilizados atingiram a média de *recalls* de 21,09% e 33,82% para C/C++ e Java, indicando que é perdido uma grande porção de vulnerabilidades, geralmente causados por "inícios desalinhados" (*misaligned source/sink definitions*) baseadas em análise manual (LI et al., 2026). Porém, assim como Hou et al. (2025), Li et al. (2026) também conclui que métodos de análise estática baseado em LLMs identificam mais vulnerabilidades únicas do que métodos tradicionais.

Com estes resultados, este cenário apresenta potencial em analisadores estáticos com *frameworks* dinâmicos para sistemas com riscos de vulnerabilidades relacionada à linguagem C/C++.

1.3 Hipótese

A hipótese desta pesquisa busca comprovar que a geração automatizada de regras por meio de um *framework* multi-agente baseado em LLMs, integrado ao motor do SemGrep, produzirá taxas de detecção (*Recall* e *F1-Score*) de vulnerabilidades (CWEs) em C/C++ estatisticamente superiores e com maior identificação de *corner-cases* quando comparada aos conjuntos de regras predefinidas dos analisadores estáticos tradicionais.

1.4 Objetivo

Os objetivos desta pesquisa estruturam-se em duas frentes: o objetivo geral, focado em solucionar o problema central do estudo, e os objetivos específicos, que descrevem as etapas práticas e teóricas indispensáveis para a realização desse propósito maior.

1.4.1 Objetivo Geral

O objetivo geral deste trabalho é demonstrar que um *framework* de síntese automatizada de regras, integrando Modelos de Linguagem de Larga Escala e integrado à ferramenta SemGrep, apresenta precisão e *recall* estatisticamente superiores às abordagens estáticas tradicionais na detecção de vulnerabilidades críticas em linguagem C/C++. Busca-se demonstrar que este modelo supera abordagens estáticas tradicionais na identificação precisa de, no mínimo, cinco categorias de vulnerabilidades de software, em inglês *Common Weakness Enumeration* (CWE).

1.4.2 Objetivos específicos

Sobre os objetivos específicos, temos:

1. Quantificar as taxas de falsos positivos e falsos negativos geradas por regras sintetizadas via LLMs em comparação com os *rulesets* predefinidos do ecossistema SemGrep.
2. Identificar e categorizar os limites de detecção semântica (*corner-cases*) superados pela abordagem gerativa na validação de vulnerabilidades.
3. Avaliar quantitativamente a precisão e revocação e *F1-Score* das regras geradas pela LLM, estabelecendo uma análise comparativa direta com os resultados de analisadores estáticos tradicionais.
4. Avaliar a eficácia da geração de regras utilizando modelos de linguagem de menor escala (com contagem reduzida de parâmetros), visando garantir a viabilidade

técnica da execução do *framework* em infraestruturas locais com restrições de custo e *hardware*.

2 Revisão Bibliográfica

Este capítulo tem como finalidade estabelecer o embasamento teórico e tecnológico que sustenta a pesquisa proposta. Inicialmente, descreve-se a metodologia utilizada para a seleção dos trabalhos consultados. Na sequência, o referencial teórico explora os paradigmas da análise estática assistida por inteligência artificial, as limitações da análise dinâmica e os desafios inerentes à análise estática tradicional. Os fundamentos técnicos são apresentados com ênfase em Modelos de Linguagem de Grande Escala (LLMs), Processamento de Linguagem Natural e os princípios de Análise Estática. Por fim, conduz-se uma revisão dos trabalhos correlatos, identificando as lacunas que esta pesquisa busca preencher.

2.1 Metodologia da pesquisa bibliográfica

A pesquisa bibliográfica foi conduzida de forma sistemática para identificar trabalhos científicos relevantes sobre a utilização de LLMs no suporte à análise estática. As buscas foram centralizadas no motor Google Acadêmico (*Google Scholar*), devido à sua ampla indexação, com foco principal em bases de dados consolidadas na área da Computação, como arXiv, IEEE Xplore e ACM Digital Library.

Para as buscas, foram definidas palavras-chave em inglês, tais como: *"static analysis"*, *"LLM"*, *"large language model"*, *"GPT"*, *"C"*, *"C++"*, *"rule generation"*, *"Linux kernel"*, *"pointer"*, *"undefined"*, *"memory"*, *"value-flow"* e *"SemGrep"*. Estes termos foram combinados por meio de operadores booleanos (*AND*, *OR*) para refinar os resultados, culminando na *string* de busca apresentada na Figura 3:

```
"static analysis" AND ("LLM" OR "large language model" OR "GPT") AND ("C" OR "C++" OR
"Linux kernel") AND ("pointer" OR "undefined" OR "memory" OR "value-flow") AND
("SemGrep" OR "semgrep")
```

Figura 3 – *Query* utilizada durante a pesquisa de materiais.

Fonte: Elaborado pelo autor (2026)

Como critérios de inclusão, estabeleceram-se: (a) trabalhos publicados entre 2021 e 2026, de forma a capturar o estado da arte; (b) disponibilidade nos idiomas inglês ou português; e (c) relação direta com o tema, priorizando artigos que contivessem implementações ou experimentos aplicados a utilização de LLMs para geração de regras para a linguagem C/C++.

Em contrapartida, os critérios de exclusão adotados foram: (a) trabalhos publicados antes de 2021, com exceção de referências clássicas ou seminais indispensáveis para a teoria;

- (b) artigos sem alinhamento direto com análise estática ou geração de regras de código; e
- (c) duplicidades de publicações entre as bases de dados consultadas.

O processo de seleção ocorreu em duas etapas. Na primeira fase (triagem), a aplicação da *string* de busca retornou 4.770 resultados, dos quais foram selecionados 47 artigos potencialmente relevantes após a leitura prévia de títulos e resumos. Na segunda fase (afunilamento), os trabalhos restantes passaram por uma leitura criteriosa da introdução e do escopo, sendo classificados em uma escala de relevância de 0 a 10 em relação aos objetivos desta pesquisa.

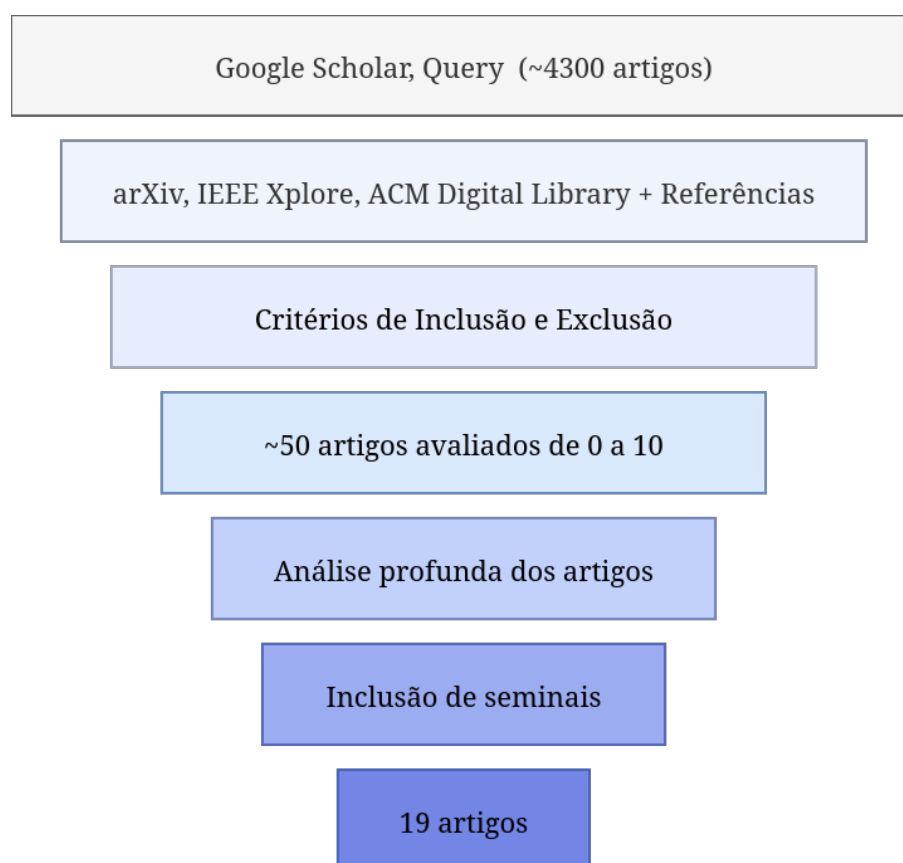


Figura 4 – Funil de pesquisa.

Fonte: Elaborado pelo autor (2026)

A avaliação de 0 a 10 foi a partir de seu resumo, palavras-chave, tecnologia aplicada. Caso o trabalho possui similaridades com outros artigos bem avaliados é considerado como suporte ou alta relevância. Esse processo de filtragem resultou em 6 artigos classificados com notas entre 9 e 10 (alta relevância), além de 9 artigos categorizados como material de suporte, totalizando 15 trabalhos contemporâneos para compor a fundamentação teórica. Adicionalmente, foram incorporados 4 trabalhos seminais anteriores a 2021, essenciais para a consolidação dos conceitos clássicos abordados neste estudo.

2.2 Fundamentos

Esta seção detalha os conceitos técnicos e contextualização diretamente empregados na implementação do projeto.

2.2.1 Análise Estática Tradicional

Verificações estáticas, segundo Aho et al. (2007), são checagens de consistência realizadas durante o processo de compilação, sendo possível capturar erros de programação cedo, antes da sua execução.

Incluindo dois tipos de análises: **(i) Verificação Sintática**, envolve a aplicação de restrições que não são aplicadas apenas pelas gramáticas, como um identificador ser declarado apenas uma vez dentro do escopo (AHO et al., 2007). **(ii) Verificação de Tipos**, assegura que as regras de tipagem da respectiva linguagem sejam seguidas, checando se os operadores ou funções estão sendo aplicados ao número ou tipo correto de operando. Se a conversão de tipos é necessária, o verificador pode inserir o operador correto na árvore de sintaxe para a conversão (AHO et al., 2007).

Não é importante somente para otimização de código, as mesmas técnicas podem ser usadas para analisar *software* já existentes para diversos padrões comuns de erros de programação (AHO et al., 2007). PREfix e Metal são duas ferramentas mencionadas por Aho et al. (2007) utilizam análise estática interprocedural, rastreando o fluxo de informações em um programa inteiro, para encontrar erros de programação estaticamente em grandes bases de código. Estas ferramentas são capazes de rastrear o caminho de entradas fornecidas pelos usuários enquanto essas *strings* são copiadas ou combinadas ao longo de vários procedimentos, acompanhando as entradas é possível prevenir contra *SQL Injection* e *Buffer Overflows* (AHO et al., 2007).

2.2.1.1 Árvore Sintática Abstrata

Durante o processo de análise sintática, é comum a geração de uma árvore de análise (*Parse Tree*), também conhecida como árvore de sintaxe concreta, onde os nós internos representam os símbolos não-terminais definidos pela gramática da linguagem (AHO et al., 2007). Contudo, para as fases subsequentes de compilação ou de análise estática de código, essa representação contém detalhes puramente estruturais que não afetam a semântica do programa (AHO et al., 2007).

A Árvore Sintática Abstrata, do inglês *Abstract Syntax Tree* (AST), surge como uma representação intermediária otimizada. Em uma AST, cada nó interno representa um operador ou um constructo de programação, enquanto nós filhos representam os operandos ou os componentes semânticos daquele constructo (AHO et al., 2007). Nós auxiliares exigidos pela gramática para contornar ambiguidades ou definir precedência

(como produções única ou parênteses) são descartados, resultando em uma estrutura de dados mais limpa e focada no significado das operações (AHO et al., 2007).

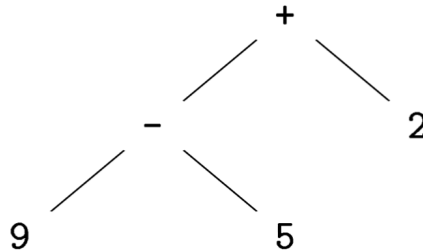


Figura 5 – Árvore Sintática Abstrata para a expressão 9-5+2

Fonte: (AHO et al., 2007)

Tomando como exemplo, na figura 5, a expressão matemática $9 - 5 + 2$, a raiz da AST representaria o operador $+$, tendo como filho esquerdo a subexpressão correspondente a $9 - 5$ e como filho direito a constante 2 . Esse agrupamento estrutural reflete a precedência e a associatividade à esquerda dos operadores, tornando redundante a presença de nós sintáticos adicionais. Em ferramentas de análise estática voltadas para linguagens complexas como C/C++, operar sobre a AST é essencial, pois permite que a verificação de propriedades e a busca por vulnerabilidades sejam conduzidas diretamente sobre a lógica estrutural do código, isolando a análise de variações gramaticais irrelevantes (AHO et al., 2007).

No contexto da análise de vulnerabilidades, a AST é fundamental para identificar padrões estruturais perigosos. Ao invés de realizar buscas textuais limitadas, os analisadores percorrem a árvore em busca de composições específicas de nós. Por exemplo, para detectar um risco de desreferenciação de ponteiro nulo (*Null Pointer Dereference*), uma regra de análise pode inspecionar a AST em busca de um nó que represente uma chamada de alocação de memória, como a função `devm_kzalloc`, verificando se não existe um nó condicional adjacente encarregado de checar o valor de retorno antes de sua utilização.

2.2.1.2 Grafos de Fluxo de Controle

Complementar à Árvore Sintática Abstrata (AST), utilizam-se os grafos de fluxo de controle, do inglês *Control Flow Graphs* (CFG). Enquanto a AST foca em representar hierarquicamente a sintaxe estrutural do código, o CFG é um grafo direcionado construído para modelar os possíveis caminhos de execução do programa, sendo a base essencial para análises sensíveis ao fluxo (*flow-sensitive*) (AHO et al., 2007).

Representação em grafo de código intermediário é muito útil para a geração de código (AHO et al., 2007). De acordo com Aho et al. (2007), essa representação é construída em dois passos, primeiro, é particionado o código intermediário em blocos básicos (*basic blocks*), que atuam como os nós do grafo. Um bloco básico é uma sequência máxima de instruções

consecutivas de três endereços onde o fluxo de controle entra obrigatoriamente pela primeira instrução, sem saltos para o interior dele, e sai apenas pela última, sem interrupções ou ramificações em seu interior (AHO et al., 2007). As arestas relacionados do grafo indicam as transições de controle (*control flow*) permitidas no programa, representando que a execução pode fluir do final de um bloco básico específico para o início de outro (AHO et al., 2007).

Na construção de um CFG, as instruções de desvio (*jumps*) endereçadas a linhas ou rótulos específicos são substituídas por referências diretas aos blocos básicos correspondentes. Essa abstração é fundamental na construção de compiladores, pois permite que otimizações subsequentes alterem livremente o conteúdo interno dos blocos sem invalidar os alvos dos desvios de controle (AHO et al., 2007). Caso não houvesse essa otimização, seria necessário corrigir os alvos toda vez que uma instrução fosse alterada.

CFGs podem ser representados na programação por qualquer estrutura de dados tradicional para grafos. Quanto ao conteúdo interno dos nós (blocos básicos), Aho et al. (2007) afirma que eles podem ser representados por um ponteiro para a primeira instrução e uma contagem de instruções. No entanto, como as otimizações alteram frequentemente a quantidade de instruções, costuma ser mais eficiente representar o conteúdo de cada bloco básico, ou nó, como uma lista encadeada (*linked list*) de instruções (AHO et al., 2007).

2.2.1.3 Análise de Fluxo de Dados

A Análise de Fluxo de Dados, em inglês *Data Flow Analysis* (DFA), é definida como um conjunto de técnicas que derivam informações sobre o fluxo de dados ao longo dos possíveis caminhos de execução de um programa (AHO et al., 2007). Tradicionalmente fundamentais para as otimizações globais em compiladores, essas técnicas permitem determinar, por exemplo, se duas expressões avaliam para o mesmo valor em todos os caminhos possíveis ou se o resultado de uma atribuição nunca é utilizado (código morto) (AHO et al., 2007).

A execução de um programa é modelada como uma série de transformações do estado do programa, onde o estado consiste nos valores de todas as variáveis do programa em um dado momento (AHO et al., 2007). Cada instrução executada transforma um estado de entrada em um novo estado de saída. No entanto, Aho et al. (2007) afirma que é impossível acompanhar todos os estados exatos devido ao número ilimitado de caminhos de execução. Por esse motivo, a DFA opera através de abstrações, ignorando detalhes irrelevantes e guardando apenas as características essenciais para o objetivo da otimização (AHO et al., 2007).

É associado a cada ponto do programa um valor de fluxo de dados (que representa a abstrações dos estados possíveis) pertencente a um "domínio" (AHO et al., 2007). O problema de DFAs consiste em encontrar uma solução para dois conjuntos de restrições principais: **Funções de transferência** (*Transfer Functions*), são restrições baseadas na semântica de cada instrução, pegando o valor do fluxo de dados antes da instrução e geram

um novo valor de saída (AHO et al., 2007). O fluxo pode acontecer "para frente" (*forward*), onde a entrada molda a saída, ou "para trás" (*backward*), onde a direção desse fluxo é invertida. **Restrições de Fluxo de Controle** (*Control-Flow Constraints*), definem como a informação transita de um bloco básico para outro dentro do CFG. Essencialmente, o valor do fluxo de dados que entra em uma instrução é matematicamente determinado pela combinação dos valores que saem das instruções ou blocos predecessores (AHO et al., 2007).

No contexto da segurança de *software* e da detecção de falhas abordadas neste trabalho, a abstração fornecida pela DFA é o motor principal que permite analisadores estáticos avançados identificarem vulnerabilidades complexas. É através do rastreamento do estado das variáveis pelos caminhos de controle que se torna viável detectar comportamentos críticos em linguagens como C/C++, como *Null Pointer Dereference* (CWE-476) ou uso de variáveis não inicializadas (CWE-457).

2.2.1.4 Sensibilidade de Fluxo

A precisão de uma análise estática depende fortemente de sua sensibilidade ao fluxo de controle. Segundo Aho et al. (2007), as abordagens de análise (como a análise de apontamento de ponteiros) podem ser categorizadas em dois tipos principais:

Análise Sensível ao Fluxo, em inglês *Flow-Sensitive Analysis* (FSA), essa abordagem acompanha o fluxo de controle do programa e computa o estado exato (por exemplo, para onde cada variável pode apontar) após a execução de cada instrução individual Aho et al. (2007). Isso significa que a análise avalia as restrições e funções de transferência passo a passo; ela não apenas considera novas informações e relações que um comando gera (*generate*), mas também contabiliza as informações anteriores que o comando invalida ou mata (*kills*).

Análise Insensível ao Fluxo, em inglês *Flow-Insensitive Analysis* (FIA), em contrapartida, essa abordagem ignora o fluxo de controle, assumindo essencialmente que cada instrução do programa pode ser executada em qualquer ordem (AHO et al., 2007). A FIA computa apenas um mapa global indicando para onde cada variável possivelmente apontar em qualquer momento da execução do programa. Em uma análise insensível ao fluxo, uma atribuição nunca invalida (*kill*) relações de apontamento; ela apenas gera novas relações. Ou seja, se uma variável puder apontar para dois objetos diferentes após a avaliação de duas instruções distintas, o sistema apenas registra e acumula a informação global de que ela aponta para ambos (AHO et al., 2007).

Aho et al. (2007) explicam que a falta de sensibilidade ao fluxo enfraquece consideravelmente a precisão dos resultados da análise, uma vez que produz superaproximações do estado real do programa. No entanto, essa abstração tem o benefício direto de reduzir drasticamente o tamanho da representação dos resultados e fazer com que o algoritmo

termine de forma mais rápida, o que é frequentemente necessário ao analisar bases de código em larga escala.

2.2.1.5 Limitações da Análise Estática

[Aho et al. \(2007\)](#) explicam o problema de encontrar todos os erros de um programa é indecidível. Uma ferramenta de análise de fluxo de dados pode até ser projetada para alertar programadores sobre todas as possíveis partes do código com uma categoria específica de erros, porém se a maioria desses avisos forem falsos alarmes, os usuários não vão utilizar a ferramenta.

Por conta dessa complexidade, [Aho et al. \(2007\)](#) afirmam que os detectores práticos de erros frequentemente não são perfeitos (à prova de falhas ou *sound*) nem completos (*complete*). As abordagens tradicionais de detecção de vulnerabilidades estática geralmente codificam o conhecimento especializado como regras projetadas manualmente, com padrões e modelos de fluxo de dados, sua dependência de padrões predefinidos limita inerentemente a generalização e a cobertura para tipos de vulnerabilidades previamente não vistos ou em evolução ([LI et al., 2026](#)).

Isso significa que a análise estática pode não encontrar todos os erros no programa, e nem todos os erros relatados têm a garantia de serem erros reais, e se as ferramentas relatassem todos os erros potenciais, o volume de falsos positivos tornariam as ferramentas inutilizáveis ([AHO et al., 2007](#)).

2.2.2 Vulnerabilidades de Linguagem

O termo Vulnerabilidade (*Vulnerability*) é definido como um conjunto de condições que permite que um invasor (atacante) viole uma política de segurança explícita ou implícita ([SEACORD, 2013](#); [Software Engineering Institute, 2016](#)). Nem toda falha de segurança (*security flaw*) leva a uma vulnerabilidade ([SEACORD, 2013](#)).

[Seacord \(2013\)](#) explicam que para que uma falha torne um programa vulnerável a ataques, é necessário que os dados de entrada desse programa cruzem um limite de segurança (*security boundary*). Uma falha pode existir no código sem as pré-condições para ser explorada; por exemplo, se o programa apresentar um erro grave mas não possui permissões para ser acessado localmente, ele não viola a política de segurança e não constitui uma vulnerabilidade ativa ([SEACORD, 2013](#)).

2.2.2.1 Common Weakness Enumeration

2.2.2.2 Common Vulnerabilities and Exposures

2.2.3 Arquitetura Transformers

2.2.3.1 Linguagem Natural

Um sistema formal amplamente utilizado para modelar a estrutura constituinte em linguagens naturais é a Gramática Livre de Contexto (GLC). Também conhecidas como gramáticas de estrutura de frase (*phrase-structure grammars*), as GLCs utilizam um formalismo que é equivalente à Forma de Backus-Naur (BNF), padrão universal para a especificação da sintaxe em computação (JURAFSKY; MARTIN, 2026).

Uma GLC consiste em um conjunto de regras ou produções, onde cada uma expressa como os símbolos podem ser ordenados e agrupados, e um léxico de palavras e símbolos. Tal estrutura define uma linguagem formal, na qual existe uma separação estrita: sentenças que podem ser derivadas pelas regras são consideradas gramaticais, enquanto as que não podem são denominadas agramaticais (JURAFSKY; MARTIN, 2026).

Contudo, essa distinção rígida é uma característica das gramáticas formais e serve apenas como um modelo simplificado das linguagens naturais. Diferente das linguagens de programação, a validade e o sentido em linguagem natural dependem intrinsecamente do contexto. Na linguística computacional, o uso desses modelos para capturar tal complexidade é fundamentado na gramática generativa, visto que a linguagem passa a ser definida pelo conjunto de possíveis 'geradas' por suas regras gramaticais (JURAFSKY; MARTIN, 2026).

2.2.3.2 Modelos de Linguagens de Grande Escala

Os Modelos de Linguagem de Grande Escala (LLMs), em sua maioria, baseiam-se na arquitetura *Transformer*, originalmente projetada para tarefas sequenciais, como a tradução automática. A arquitetura consiste em dois componentes principais: o Codificador (*Encoder*), o qual converte uma entrada de *tokens* em uma sequência de vetores representativos (frequentemente chamada de estado oculto ou contexto), e o Decodificador (*Decoder*), que utiliza o contexto gerado pelo codificador para produzir iterativamente uma sequência de *tokens* de saída, um por vez (TUNSTALL; WERRA; WOLF, 2022).

Com o passar dos anos, esses blocos funcionais foram adaptados para atuar como modelos independentes. Embora existam centenas de variações de *Transformers*, a maioria se encaixa em um desses três tipos:

Apenas Codificador (*Encoder-only*): Esses modelos convertem uma sequência de texto de entrada em uma representação numérica, sendo muito eficientes para tarefas como classificação de texto ou reconhecimento de entidades nomeadas. A representação calculada para cada *token* nessa arquitetura depende tanto do contexto à esquerda (antes

do *token*) quanto à direita (depois do *token*); esse mecanismo é geralmente denominado atenção bidirecional (TUNSTALL; WERRA; WOLF, 2022).

Apenas Decodificador (*Decoder-only*): Dada uma entrada de texto, esses modelos completam a sequência prevendo iterativamente a palavra subsequente mais provável. A família de modelos GPT pertence a essa classe. Nesse caso, a representação de um *token* depende exclusivamente do contexto à sua esquerda; esse comportamento é conhecido como atenção causal ou autorregressiva (TUNSTALL; WERRA; WOLF, 2022).

Codificador-Decodificador (*Encoder-decoder*): Utilizados para modelar mapeamentos complexos de uma sequência de texto para outra (arquiteturas *sequence-to-sequence*), esses modelos são projetados para tarefas como tradução automática e sumarização de textos (TUNSTALL; WERRA; WOLF, 2022).

2.2.4 Métricas de Avaliação

2.3 Referencial teórico

O referencial teórico apresenta os conceitos fundamentais que sustentam o desenvolvimento deste projeto. Propõe-se um contraponto analítico entre os métodos da análise estática tradicional e as novas abordagens voltadas à automatização da geração de regras. A estruturação dos tópicos foi concebida para apresentar gradativamente as tecnologias envolvidas, destacando o papel das LLMs e sua relevância para a modernização das ferramentas de verificação de código na atualidade.

2.3.1 Análise estática baseada em regras

Em anos recentes, ferramentas de análise estática baseada em regras foram introduzidas. Complementando as ferramentas tradicionais, ao identificar previamente problemas durante o desenvolvimento pré-produção, reduzem significativamente o custo e complexidade de *debugging* e manutenção (JIANG et al., 2025). Dos componentes mais utilizados, um dos mais relevantes são SemGrep e CodeQL (JIANG et al., 2025; HOU et al., 2025).

Formalmente, SemGrep é uma ferramenta de análise estática que suporta estruturas complexas e verificações básicas de fluxo de dados no código, permitindo inspeções sem execução para identificar vulnerabilidades de segurança, qualidade de código e violações a conformidade (HOU et al., 2025).

Sua gramática baseada em YAML mimetiza a sintaxe da linguagem alvo, reduzindo sua curva de aprendizado ao contrário de seu concorrente CodeQL (HOU et al., 2025), que possui sua própria gramática lógica, tratando o código como um banco de dados relacional, sendo possível realizar análises profundas porém custosa em tempo de aprendizado.

SemGrep converte código em uma árvore de sintaxe abstrata universal (UAST), aplicando um *pattern-matching* uniforme em mais de 30 linguagens (HOU et al., 2025).

Com sua ativa continuidade e plataforma capaz de compartilhar regras existentes melhora sua adaptabilidade e desenvolvimento colaborativo.

```

1  patterns:
2    - pattern-inside: |
3      $FUNCDECL(..., $USER_DATA, ...) {...}
4    - pattern-either:
5      - pattern: |
6        $PB.command(..., $USER_DATA, ...)
7      - patterns:
8        - pattern-either:
9          - pattern-inside: |
10            $X = $USER_DATA; ...
11          - pattern-inside: |
12            $X = trim($USER_DATA); ...
13        - pattern: $PB.command(..., $X, ...);

```

Figura 6 – Exemplo de regra simplificada para detecção de falhas OSCI em Java.

Fonte: Adaptado de (JIANG et al., 2025).

Jiang et al. (2025) explica com a figura 6 que a partir de uma regra no SemGrep com o propósito de identificar uma falha de segurança (como a injeção de comandos no sistema operacional), do inglês *Operating System Command Injection* (OSCI), em Java, sendo possível visualizar a utilidade prática desse *framework*.

Para que a busca seja flexível e rastrear fluxo de dados, o SemGrep utiliza dois recursos fundamentais: (i) **Metavariáveis** (ex.: `$USER_DATA`, `$X`), a partir de um cifrão e nome da variável em letras maiúsculas. Essas variáveis são atribuídas quando SemGrep identifica um padrão que se encaixa, sendo "salvo" o valor na metavariável, consequentemente, se a metavariável é reutilizada na mesma regra, o SemGrep exigirá que o valor capturado seja exatamente o mesmo, o que permite *Data Flow*. (ii) **Ellipsis/Curinga** (...), representando qualquer sequência de coisas. Assim independente da quantidade de argumentos em uma função, número de linhas de código entre duas instruções, é possível omitir detalhes não importantes com (...), tornando SemGrep uma ferramenta abstraída.

Os predicados são as afirmações que procuram o código. *Tags* como *pattern* ou *pattern-inside* são predicados, com ***pattern*** sendo uma afirmação exata, como na linha 5, e ***pattern-inside*** define o contexto, não apontando a vulnerabilidade exata, mas exige que qualquer correspondência subsequente aconteça dentro da estrutura dele, como na linha 2.

A partir destes predicados, é possível combinar operações lógicas (*AND*, *OR*, *NOT*) sendo, respectivamente, *patterns*, *pattern-either*, *pattern-not*. (i) O predicado ***patterns*** representa a lógica *AND* (E), para toda regra ou condição listada abaixo deste bloco

devem ser verdadeiras simultaneamente. Como na linha 1, o bloco principal exige que o código esteja dentro da função (linha 2) **E** corresponda a um dos casos de injeção (linha 4). (ii) ***pattern-either*** representa a lógica *OR* (OU), se o código corresponde a qualquer uma das condições aninhadas sob este predicado, a avaliação será verdadeira. Na linha 4, a regra procura um fluxo direto de `$USER_DATA` ou um fluxo indireto por atribuição. (iii) ***pattern-not*** representa a lógica *NOT* (NÃO). Não presente no exemplo, porém usado para excluir falsos positivos, anulando caso a regra bater coma condição aninhada no predicado.

Com estes conhecimentos, é possível compreender mais profundamente como a ferramenta utilizada funciona. SemGrep possui maneiras de lidar com falsos positivos nativamente, nos permitindo liberdade para aperfeiçoar a análise de regras.

2.3.1.1 Árvore Sintática Abstrata

2.3.2 Aplicação de LLMs em análise estática

2.3.3 Abstração de Regras

2.4 Trabalhos relacionados

Os três trabalhos apresentados a seguir foram selecionados por enfrentarem diretamente o mesmo problema central: a análise estática tradicional, buscando melhorias através de LLMs e geração de regras utilizando a mesma ferramenta SemGrep. Cada um deles avança com seus próprios *pipelines* na utilização da LLM, cada um com um papel diferente no processo de geração de regras.

2.4.1 Trabalho 1: *KNighter: Transforming Static Analysis with LLM-Synthesized Checkers* (YANG et al., 2025)

O trabalho de Yang et al. (2025) é considerado o estudo pioneiro para a área de Analisadores Estáticos Automatizados. Publicado em SOSP 25 (*ACM SIGOPS 31st Symposium on Operating Systems Principles*), em Seul, República da Coreia. KNighter foi o primeiro sistema de geração de analisadores estáticos totalmente automatizado (YANG et al., 2025).

Os autores buscam resolver o problema principal com métodos de análise tradicional. Apesar da análise estática ser o método mais eficiente pela sua falta de execução de código, para gerenciamento de sistemas de larga escala, esses métodos falham na falta de identificar diferentes padrões de bugs e gerenciamento de bases enormes de código (YANG et al., 2025). A figura 1 mostra a motivação que os autores tinham sobre analisadores automatizados.

Motivado pelas limitações expostas, o trabalho propõe um *framework* voltado à extração de padrões a partir de histórico de *commits*. Como exemplo empírico, a identificação de chamadas à função `devm_kzalloc` desacompanhadas de verificação de nulidade configura uma heurística válida para a detecção da vulnerabilidade *Null Pointer Dereference* (CWE-476) (YANG et al., 2025). Com um *commit* como entrada, KNighter retorna como saída um analisador gerado para *Clang Static Analyser* (CSA). Sendo um verificador válido aquele que identifica o código pré alteração do *commit* como *bug* e o código corrigido como correto (YANG et al., 2025).

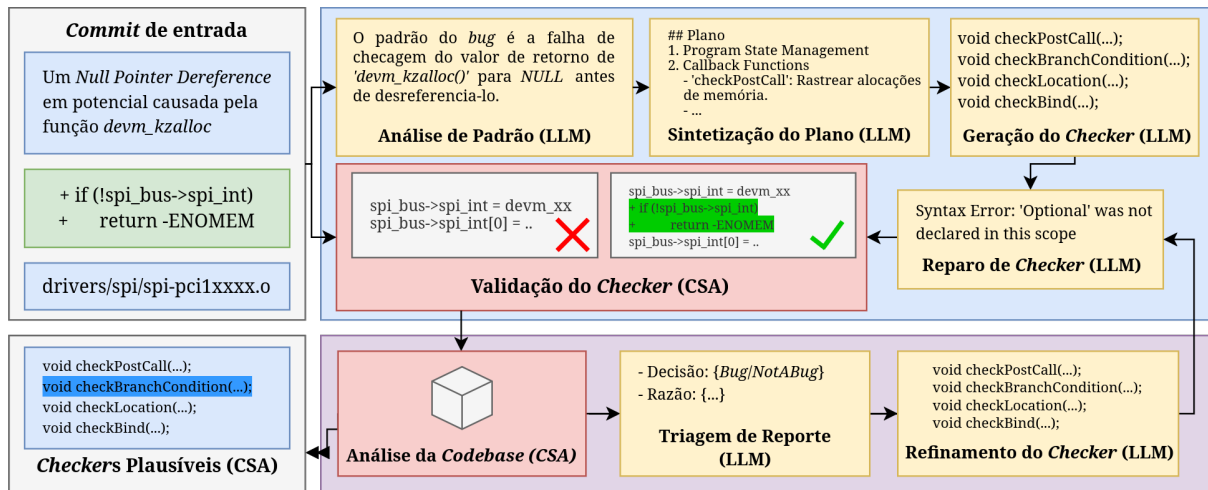


Figura 7 – Overview do KNighter

Fonte: Adaptado para o português para melhor compreensão (YANG et al., 2025).

O motor principal do framework é *Clang Static Analyser* (CSA), Yang et al. (2025) explica que CSA opera a partir de uma execução simbólica *path-sensitive*, construindo uma representação interna chamada *ExplodedGraph*. CSA é modulável pela sua construção a partir de *checkers*, sendo eles, pequenos componentes especializados, montados a partir de um *template* (YANG et al., 2025).

Na figura 7 mostra como o KNighter funciona em duas fases: sintetização de *checkers*, o projeto analisa o *patch* de entrada para identificar padrões de *bugs*, sintetiza, a partir do conteúdo identificado, um plano de detecção e implementa um CSA *checker* (YANG et al., 2025). Se ocorre qualquer erro de compilação, é encaminhado para um agente de sintaxe que automaticamente repara o *checker* a partir das mensagens de erro (YANG et al., 2025).

No que tange à implementação do trabalho, a metodologia descrita por Yang et al. (2025) fundamenta-se na coleta de *commits*, viabilizando uma análise comparativa e estrutural entre versões vulneráveis de código-fonte e suas respectivas correções. Foram coletados 10 tipos de CWEs, criando para cada CWE um respectivo CSA *checker*. Além dessa coleção, foram preparados 3 tipos de exemplos de *checkers* para aprendizado *in-context*, lidando com os respectivos CWEs: *Null Pointer Dereference* (CWE-476), *Use*

of *Uninitialized Variable* (CWE-457) e *Double Free* (CWE-415) (YANG et al., 2025). Além desses exemplos, os autores identificaram repetições de atividades que cada *checker* reutiliza, então separaram em 9 *sub-checkers* para poderem serem reutilizados em outros verificadores.

Para a avaliação de cada *checker*, é analisado sua eficácia para o *bug* do *commit* analisado e reconhecer sua correção. No repositório original do *patch*, o analisador realiza uma varredura integral na versão do código imediatamente à correção, quantificando o total de vulnerabilidades identificadas (N_{buggy}). Em seguida, uma nova varredura é executada na base atualizada para contabilizar as vulnerabilidades remanescentes (N_{patched}), viabilizando assim a validação da eficácia do verificador. Um *checker* é considerado válido se $N_{\text{buggy}} > N_{\text{patched}}$ e $N_{\text{patched}} < T_{\text{valid}}$, onde T_{valid} é um valor limite (padrão 50) (YANG et al., 2025).

Apesar de seu forte pioneirismo, Yang et al. (2025) analisa suas falhas com seu projeto, com 36,06% dos *commits* não sendo possível gerar um analisador a partir disso, com sua maioria falhando por implementações incorretas, com o problema comum são otimizações realizadas pelo respectivo compilador (como *strcpy* e *memset*), impedindo que o *checker* lidasse com essas chamadas. Além disso, notaram limitações relacionadas a *Buffer Overflow* e *Use of Uninitialized Variable*, quais os autores acreditam que a razão é por dois fatores principais: a análise estática inerentemente possui limitações com determinar valores precisos, principalmente quando é preciso determinar limites de *buffers*, e sua dificuldade para análises de código *multi-thread*, como gerenciar uso correto de *locks* (YANG et al., 2025).

KNightter relata dentre 32 *checkers* plausíveis, com 16 não identificando novas falhas, foram relatados 90 novos *bugs*, que depois de uma inspeção manual, 32,2% foram falsos positivos (YANG et al., 2025). Sendo o principal erro envolvem os analisadores com seu padrão correto e regras para *matching* corretos, porém o *checker* falha em gerenciar *triggers* ou a condição do estado corretamente (como identificar um ponteiro já verificado antes de seu uso) (YANG et al., 2025).

Para o presente trabalho, a contribuição de Yang et al. (2025) é insubstituível no ponto de vista conceitual: a automatização para um sistema de análise estática, gerenciamento de mais de um agente para refinamento e correção, cobrindo CWEs relevantes adotados neste TCC. No entanto, a ferramenta auxiliar difere: enquanto Yang et al. (2025) opera gerenciando regras para CSA, este trabalho opera exclusivamente com SemGrep, com suas regras de inclusão e exclusão lógica auxilia para evitar falsos positivos, que foi um defeito crítico em KNightter. Além disso, KNightter utiliza somente *commits* para seu treinamento, o que limita sua detecção pelo simples fato de existirem outras vulnerabilidades que não se encaixam nos *patches* já corrigidos, então para contribuir na solução deste problema, a utilização de *datasets* como NSA Center for Assured Software (2017) e *dataset* personalizado de Li et al. (2026).

2.4.2 Trabalho 2: *Generation, Migration and Optimization of Cross-Language Static-Analysis Rules Based on Large Language Models* (HOU et al., 2025)

O trabalho de Hou et al. (2025), publicado no IEEE International Conference on Software e Quality, Reliability and Security (QRS) em 2025, representa um dos trabalhos que compõem o atual estado da arte em Análise Estática Automatizado até o momento da elaboração desse TCC. Com a motivação de melhorar a análise estática para *bugs* contextualizados de específicos projetos, utilizando *rulesets* lidando com o máximo de casos (*corner-cases*) diminuindo o trabalho braçal e capacidade de migrar regras para diferentes linguagens (HOU et al., 2025).

Os autores citam KNightier, explicando que apesar de sua efetividade, KNightier não possui dinâmica para lidar com diferentes linguagens, exigindo diversas alterações manuais para conseguir seu funcionamento (HOU et al., 2025). Propondo um novo método baseado em LLMs que gera regras de análise estática a partir de padrões de históricos de código, utilizando regras gramáticas e *few-shot learning* em contexto, além de suporte para migração de regras existentes para outras linguagens, geração de exemplos para cada regra especificado para lidar com *corner-cases* (HOU et al., 2025).

Arquiteturalmente, o *framework* proposto adota uma estratégia de *few-shot learning* para administrar conhecimento do domínio adquirido das LLMs durante seu pré-treinamento não supervisionado, integrando especificações de sintaxe de regras, histórico de informações de erros e descrição de otimizações de regras para guiar o comportamento de geração de regras das LLMs (HOU et al., 2025). Aponta Hou et al. (2025) que isso efetivamente soluciona dificuldades em *corner-cases* que seriam mais complexos para um humano criar, diminuindo o caso de falsos positivos e falsos negativos.

A ferramenta de análise estática utilizada, para reduzir o uso de apenas LLMs, é SemGrep. Os autores explicam que análise estática baseada em regras proveem extensibilidade por suas regras customizáveis, permitindo checagens para problemas específicos para projetos específicos (HOU et al., 2025). Consequentemente, Hou et al. (2025) afirma que não existe necessidade da construção de um analisador específico para realizar essas checagens. Para o objetivo do artigo com migração de regras, SemGrep se encaixa perfeitamente por seu suporte a mais de 30 linguagens, mimetizando qualquer das linguagens suportadas com seu arquivo YAML (HOU et al., 2025).

Do ponto de vista técnico, a abordagem proposta segue um *workflow* de três estágios: a Geração, Migração e Otimização das regras. No estágio inicial de Geração, o método dos autores é reconhecimento de *patches* de código e gera descrições das regras, assim gera a regra a partir da descrição. Com as regras geradas, cada uma é avaliada em *snippets* de teste para verificar uma eficácia, então as descrições das regras podem ser otimizadas de acordo com falsos positivos e falsos negativos (HOU et al., 2025). Para o segundo estágio,

seu método gera descrições de linguagens naturais e seus *snippets* de teste para a migração da regra, subsequentemente criando a regra. No último estágio, seu método lê a regra criada, independente se foi gerada ou não, e cria *snippets* para lidar com *corner-cases*, seguindo os mesmos passos do estágio um (HOU et al., 2025).

Sua construção de metodologia segue um fluxo similar ao Yang et al. (2025), apesar de criar uma avaliação extra relacionada à migração de linguagens. Para a utilização de *datasets* para treinamento, Hou et al. (2025) utiliza, para geração de regras, foi selecionado aleatoriamente 111 regras dentre o *ruleset* oficial do SemGrep, cada regra contendo um YAML e *snippets* de um código com *bugs* e sua correção, inserindo cada regra para seu agente realizar a descrição de cada regra, com uma validação manual para garantir que as descrições correspondiam a cada regra. Para a adaptação de gramática do SemGrep, Hou et al. (2025) utiliza a documentação oficial do SemGrep, incluindo partes relacionadas a Sintaxe de Regras e Sintaxe de Padrões. Convertendo o texto para *Markdown*, e inserindo a gramática no contexto dos agentes. Apesar de a documentação ser feita para leitores humanos, Hou et al. (2025) inseriram códigos exemplos para um treinamento *few-shot* para sua familiarização com a gramática.

Para validar seu experimento, Hou et al. (2025) utilizam três validações. **Validação Gramática**, regras com a gramática válida são aquelas que a regra gerada é reconhecida pela ferramenta de análise estática, para essa avaliação, foi aplicado cada regra com uma amostra de código aplicado na ferramenta, e examinado se SemGrep relata algum erro de gramática para a regra (HOU et al., 2025); **Validação Funcional**, uma regra é funcionalmente válida a partir que as regras geradas conseguem reportar *bugs* em uma amostra de teste. Para cada amostra de teste, contendo N *bugs* que se encaixam na respectiva regra, se a regra produzida retorna B_{VP} verdadeiros positivos e B_{FP} falsos positivos, é definido dois indicativos para avaliar a validade da regra: (HOU et al., 2025)

$$Taxa de Falsos Positivos : FP = \frac{B_{FP}}{N} \quad (2.1)$$

$$Taxa de Verdadeiros Positivos : TP = \frac{B_{VP}}{N} \quad (2.2)$$

Utilizando essa metodologia de avaliação, Hou et al. (2025) utiliza o trabalho de Yang et al. (2025) como comparação de eficácia. Como o trabalho de KNightter provê publicamente um *dataset* com 61 *commits* de *bugs fixes* no Kernel Linux, contendo o erro e sua correção, Hou et al. (2025) avalia seu método para, além de comparar com KNightter mas, avaliar sua performance no Kernel Linux.

Resultados de Hou et al. (2025) criaram 41 regras válidas comparadas com as 37 geradas por Yang et al. (2025), modestamente ultrapassando performance contra KNightter, apesar dos autores afirmarem que seu trabalho precisa de mais refinamento. Além de comparar com KNightter, Hou et al. (2025) compara sua efetividade contra ferramentas comuns de análise estática como Smatch, CPPCheck, *ruleset* oficial do SemGrep e TScanCode.

Nenhuma das ferramentas identificaram os mesmos de alguns *bugs* que sua metodologia conseguiu, que os autores dizem ser talvez pela falta de conhecimento do domínio específico do projeto analisado que métodos baseados em LLMs conseguem detectar (HOU et al., 2025).

Para o presente trabalho, o estudo de Hou et al. (2025) cumpre dois papéis fundamentais. Primeiro, apresenta aprimoramentos comparado com o seu pioneiro KNighter com seu fluxo lógico interpretativo, se mostrando promissor utilizando ferramentas sem limites de linguagem, Hou et al. (2025) mostra que SemGrep, com suas funções lógicas, auxilia para evitar falsos positivos, apesar de seu custo computacional pelos diversos agentes. Segundo, estabelece o referencial na utilização de ferramentas tradicionais para integração com LLMs. Mesmo sem integração de outros *datasets* para inclusão de novas regras, Hou et al. (2025) foi capaz de exceder em performance pela mudança de *workflow* para cada *commits*.

2.4.3 Trabalho 3: *LLM-based Vulnerability Detection at Project Scale: An Empirical Study* (LI et al., 2026)

O trabalho de Li et al. (2026), publicado no arXiv em 27 de janeiro de 2026, aguardando a revisão de seu trabalho para ser publicado em IEEE Transactions on Software Engineering (TSE), dentre os artigos encontrados, o trabalho mais atual na área de análise estática de vulnerabilidades automatizados via LLMs.

Sua metodologia foi pensada para preencher as seguintes lacunas na análise estática automatizada: **(1) Efetividade e Complementaridade em projetos de mundo real**, métodos para fonte de regras possuem limitações em personalização. Não se tornando claro sua eficácia quão generalizado e robusto são de fato esses métodos, quanto aplicados em projetos capazes de conter diversos tipos de vulnerabilidades (LI et al., 2026); **(2) Análise Abrangente sobre Causa de Raízes de Falhas**, Li et al. (2026) afirma que estudos recentes de análise estática baseada em LLMs priorizam métricas como precisão, *recall* e *F1-Score*, sem analisar sistematicamente os fatores que afetam seu uso prático em um projeto no mundo real. Em particular, as raízes de casos de falsos positivos são raramente categorizadas ou quantificadas, dificultando usuários em como melhorar as ferramentas (LI et al., 2026); **(3) Escalabilidade e *Overhead***, os autores explicam que uma das principais preocupações nesses métodos baseado em LLMs é seu custo e eficiência. Porém, estudos recentes raramente medem o potencial custo máximo do uso computacional desses analisadores, como contagem de *tokens* e tempo de processamento dependendo de projetos (LI et al., 2026).

A partir dessas lacunas, o trabalho de Li et al. (2026) apresenta um estudo empírico analisando 5 projetos de análise estática baseado em LLMs lidando com múltiplos tipos de CWEs, linguagens de programação e *workflows*. Sua avaliação utiliza duas configurações

complementares: (i) *dataset* personalizado com 222 vulnerabilidades conhecidas em contexto de mundo real, cobrindo 8 tipos de CWEs em C/C++ e Java, utilizado para avaliar a detecção de vulnerabilidades sistematicamente (LI et al., 2026). (ii) 24 projetos *open-source* ativos, utilizados para avaliar em usabilidade prática no mundo real.

Em seguida, foi examinado manualmente 385 resultados de análises, com mais de 150 horas para identificar cada falso positivo e sua causa raiz, e medir o seu custo máximo ou *overhead* (LI et al., 2026). Primeiro resultado de Li et al. (2026) foi que **métodos existentes baseados em LLMs exibem cobertura limitada para vulnerabilidades de mundo real**, utilizando seu *dataset* personalizado, os métodos avaliados atingiram um *recall* médio de **21,09%** e **33,82%** em C/C++ e Java, respectivamente, indicando que eles ainda falham em uma grande porção de vulnerabilidades, geralmente pela falta de contexto e definição contínua da variável. Apesar dessa conclusão, os autores afirmam que esses métodos, comparado com ferramentas tradicionais, ainda descobrem mais vulnerabilidades únicas (LI et al., 2026).

Segunda conclusão foi **ambos os métodos de análise estática (tradicionais e baseada em LLMs) exibem uma alta taxa de falsos positivos**, com a melhor ferramenta analisada ainda teve **85,3%** de taxa média de falsos positivos (LI et al., 2026). Dentre a avaliação das 385 detecções, foi estudado a razão de cada falso positivo e quantificado sua distribuição dentre ferramentas, mostrando que raciocínios superficiais de fluxo de dados, a identificação imprecisa de fontes e negligência de pontos críticos do programa em contextos complexos constituem as causas-raiz predominantes para falsos positivos analisados (LI et al., 2026). A última conclusão é **análise de detecção de vulnerabilidades baseada em LLMs para projetos de larga escala continua computacionalmente caro**, Li et al. (2026) realizou uma análise de custo em que mostrou que se pode custar entre cem mil *tokens* à cem milhões de *tokens* por projeto (p. ex., até 38,31 milhões de tokens de entrada e 38,07 milhões de tokens de saída), além do processamento de múltiplas horas para múltiplos dias (p. ex., até 4.368 minutos), criando uma limitação crítica para sua escalabilidade.

Com o foco na linguagem C/C++, Li et al. (2026) utilizou KNightier de Yang et al. (2025) e RepoAudit de Guo et al. (2025) e para métodos tradicionais, utilizaram SemGrep (*ruleset* padrão) e CodeQL.

Considerando as seguintes métricas de avaliação, **Com o *dataset* personalizado**, é considerado uma vulnerabilidade detectada com sucesso pela ferramenta enquanto ela reporta uma vulnerabilidade (ou seja, positivo verdadeiro: TP), caso contrário, a vulnerabilidade é despercebida pela ferramenta (ou seja, falso negativo: FN) (LI et al., 2026). Em particular, já que códigos não vulneráveis nos projetos são imensos e não totalmente identificado, Li et al. (2026) foca em *recall* do que precisão.

$$Recall = \frac{TP}{TP + FN} \times 100\%. \quad (2.3)$$

Dentre os projetos *open-source* de mundo real, é desconhecido o número de vulnerabilidades presentes. Logo, Li et al. (2026), para cada projeto, foi capturado: (i) **#Reportes**, o total de número de avisos produzidos, (ii) **#Arquivos**, o número de arquivos fontes distintos que contém pelo menos um aviso, e (iii) **#Amostras FPs**, o número de falsos positivos identificados pelas ferramentas dentre os avisos. A partir desses títulos, foi estimado a taxa de detecção de falsos positivos em amostras, do inglês *Sampled False Discovery Rate* (SFDR) como:

$$SFDR = \frac{\#AmostrasFPs}{\#Reportes}. \quad (2.4)$$

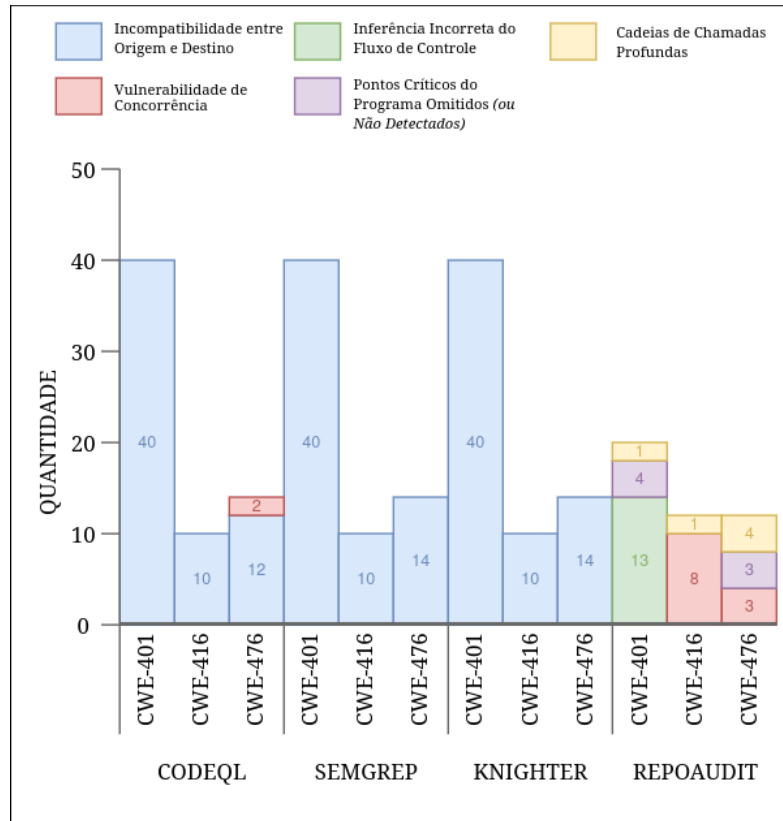


Figura 8 – Análise de CWEs (*Memory Leak, Use After Free, Null Pointer Dereference*)

Fonte: Adaptado para o português para melhor compreensão (LI et al., 2026).

Dentre essas ferramentas, RepoAudit ganhou destaque sendo a ferramenta com melhor *recall* dentre todas as vulnerabilidades de C/C++ avaliadas. Superando CodeQL, SemGrep e KNighter em todas as CWEs analisadas (LI et al., 2026). Após a avaliação manual dos autores, Li et al. (2026) mostra na 8 que a maioria das faltas de detecções são de identificação de fontes e APIs incorretos ou incompletos.

Para o presente TCC, o trabalho de [Li et al. \(2026\)](#) é essencial para a definição das fraquezas dos métodos implementados até o momento. Com sua análise profunda nas razões de falsos positivos, comparando com métodos tradicionais e baseados em LLMs, [Li et al. \(2026\)](#) revela um padrão no desafio de minimizar falsos positivos, onde tanto métodos tradicionais e baseados em LLMs possuem. Com seus resultados, sabemos direções em quais áreas necessitam mais atenção para a minimização de detecção de erros, além de seu *dataset* essencial para aperfeiçoamento de regras.

2.4.4 Síntese comparativa

A análise individual dos três trabalhos relacionados permite, em conjunto, delinear com precisão o estado da arte em Análise Estática Baseado em LLMs e identificar lacunas que este TCC busca preencher. Para facilitar essa análise, a Tabela 2 sintetizam as características técnicas dos trabalhos relacionados entre si e o diferencial introduzido pela proposta deste trabalho.

Tabela 2 – Comparação técnica entre os trabalhos relacionados

Critério	Yang et al. (2025)	Hou et al. (2025)	Li et al. (2026)	Este trabalho
Foco principal	Geração automatizada de verificadores estáticos a partir de histórico de <i>patches</i> .	Geração, otimização e migração multilinguagem de regras de análise estática.	Estudo empírico comparativo de detectores baseados em LLMs em escala de projeto.	Geração de regras automatizada a partir de histórico de <i>patches</i> e <i>datasets</i> .
Método/Tecnologia	LLMs + Clang Static Analyser (CSA)	LLMs (<i>few-shot learning</i>) + SemGrep	Avaliação de múltiplos agentes/geradores LLM frente a analisadores tradicionais.	LLMs (<i>few-shot learning</i>) + <i>CWEs datasets</i> + SemGrep
Métrica Principal de Avaliação	Taxa de validade dos <i>checkers</i> e volume de novos <i>bugs</i> detectados.	Validade sintática/ funcional e eficácia na transposição de regras entre linguagens.	<i>Recall</i> , Taxa de Falsos Positivos e custo computacional (tempo/ <i>tokens</i>).	<i>Recall</i> , Taxa de Falsos Positivos e custo computacional (tempo/ <i>tokens</i>).
Limitação / Trabalho Futuro	Acoplamento estrito ao ecossistema C/C++ e geração de falsos positivos diante de lógicas de estado complexas.	Alta taxa de falsos positivos e dificuldade em fluxo de dados.	Taxas de falso alarme insustentáveis decorrentes de raciocínio de fluxo raso e custo de inferência proibitivos.	–

Fonte: Elaborado pelo autor (2026)

3 Desenvolvimento

Este capítulo apresenta a proposta de desenvolvimento deste trabalho, detalhando a arquitetura do sistema, as decisões metodológicas, o planejamento experimental e o cronograma de execução previstos para TCC2, expandindo como será realizada cada etapa da solução, bem como serão apresentados prazos para cada fase.

3.1 Solução proposta

Este trabalho utiliza o método *Design Science Research* (DSR), conforme definido [Hevner et al. \(2004\)](#), como método científico orientador. O DSR é um paradigma de pesquisa fundamentalmente focado na resolução de problemas, responsável por produzir quatro tipos de artefatos: constructos, modelos, métodos, instâncias ([HEVNER et al., 2004](#)). Este trabalho, é por definição, um artefato projetado para resolver um problema organizacional relevante, otimização da análise estática em projetos de larga escala, realizando uma solução computacional avaliável.

A solução proposta consiste em uma Extensão Geradora de Regras baseado em LLMs direcionada à Projetos de Larga Escala para a ferramenta de análise estática, SemGrep. O ciclo deste trabalho está estruturado em quatro fases: (1) **construção de *dataset***, organização e identificação de *datasets* públicos e banco de *commits* de bases de código de projetos *open-source*; (2) ***pipeline* de geração de regras**, aplicação de LLMs em um *pipeline* para gerar regras satisfatórias para a ferramenta SemGrep; (3) **avaliação de resultados**, analisar cada relatório para obter taxa de falsos positivos no *dataset* de aprendizado; (4) **aplicação no mundo real**, com o conjunto de regras, será analisado os resultados em projetos *open-source* como Linux Kernel.

3.1.1 Arquitetura do trabalho

Este trabalho é organizado em módulos independentes, um *dataset* global incluindo todos os CWEs selecionados e identificado, um *pipeline* de geração de regras baseado em LLMs que é capaz de gerar regras a partir de qualquer projeto e analisar os *commits* somando o conhecimento do *dataset*.

1. O módulo de *dataset* é estruturado a partir de bases de código providas por [NSA Center for Assured Software \(2017\)](#). Esse repositório fornece um volume expressivo de amostras de código categorizadas por CWEs, viabilizando o treinamento e a validação do modelo com base nas referências metodológicas estabelecidas por [Yang et al. \(2025\)](#) e [Hou et al. \(2025\)](#). Além da base de [NSA Center for Assured Software](#)

(2017), será coletado históricos de *commits* de projetos *open-source* ativos, obtendo mais exemplos de *corner-cases*.

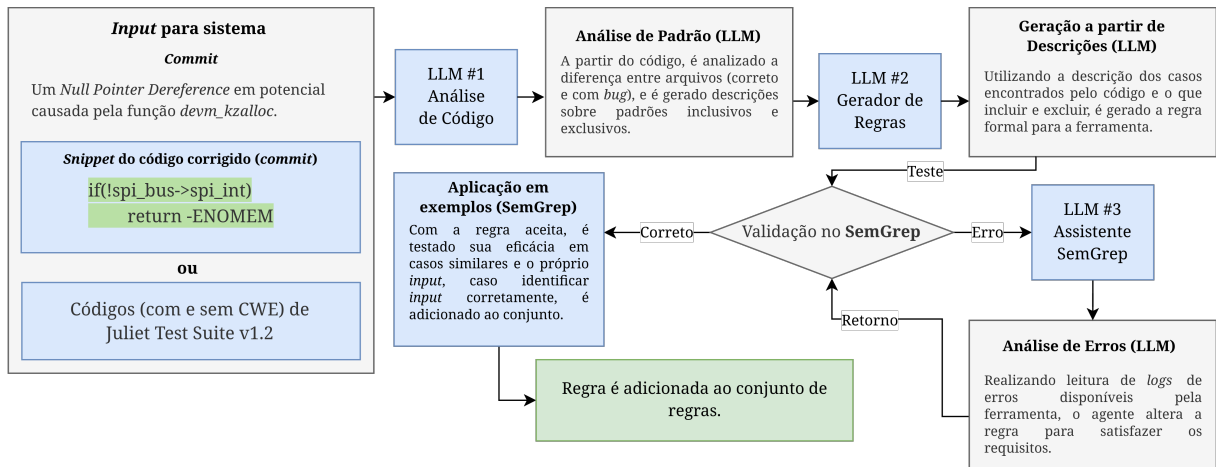


Figura 9 – Pipeline para gerador de regras.

Fonte: Elaborado pelo autor (2026)

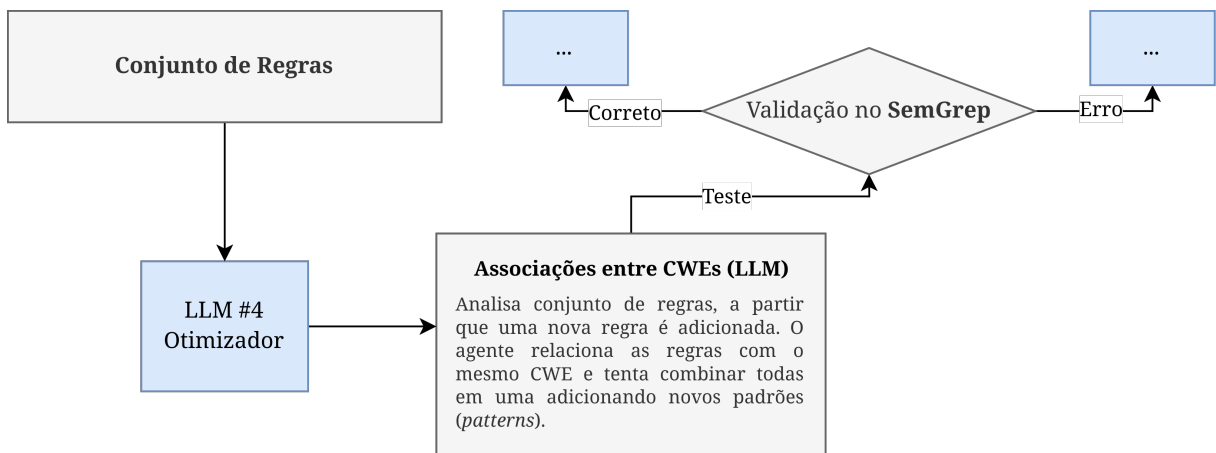


Figura 10 – Pós-processamento para otimizar regras e abranger *corner-cases*

Fonte: Elaborado pelo autor (2026)

2. A camada do *pipeline* é composta por: A partir de uma entrada contendo o código com erro e corrigido, seja de um respectivo *commit* ou de um *dataset*, é computado pelo agente responsável por analisar códigos, a partir do *input* são analisados seus padrões e o que difere o *bug* da *correção*. Com sua análise, o agente (#1) gera um reporte sobre a entrada, direcionando o próximo agente à gerar a regra. O modelo (#2), responsável por gerar a regra formal para a ferramenta, gera a regra a partir do conteúdo do reporte com o contexto da gramática do SemGrep. Com a regra gerada, é inserido a regra na ferramenta e é realizado uma validação de gramática, caso SemGrep reprovar a regra, é direcionado para o agente (#3), especializado na

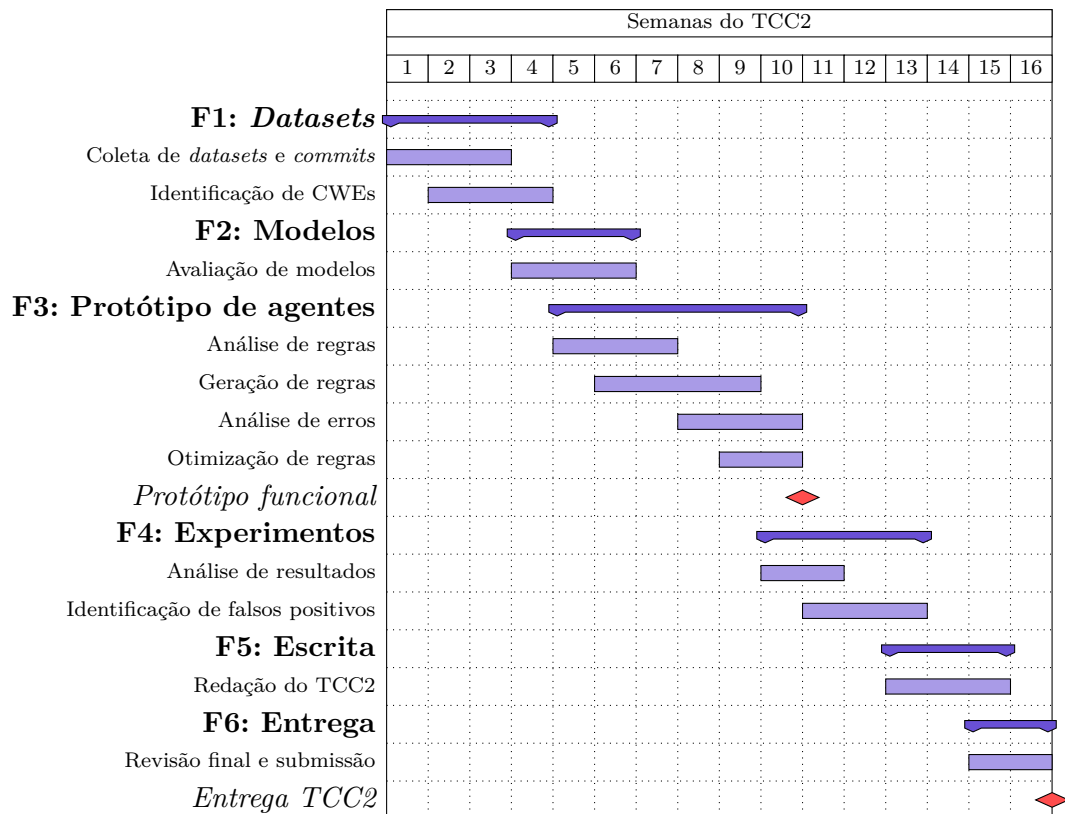
ferramenta, avalia os *logs* de erro e corrige a regra, retornando para a validação novamente. Caso a regra seja validada, é testado a regra em bancos da mesma CWE para calcular eficácia e a regra é adicionada para o conjunto de regras. Depois de adicionar a regra ao conjunto, o agente (#4) analisa todo o conjunto e unifica regras lidando com os mesmos CWEs, incluindo mais *corner-cases*.

Para todos os agentes mencionados, teremos como objetivo secundário, visto na 1.4, a utilização de modelos pequenos que podem ser usados localmente e com menos gasto de tokens.

3.2 Cronograma de trabalho

O cronograma a seguir está organizado as atividades do TCC2 em 16 semanas, conforme ilustrado no diagrama de Gantt da Figura 11.

Figura 11 – Cronograma de atividades – Diagrama de Gantt



Fonte: Elaborado pelo autor (2026)

Referências Bibliográficas

AHO, A. V. et al. *Compilers: Principles, Techniques, and Tools*. 2nd. ed. [S.l.]: Addison-Wesley, 2007. ISBN 0-321-48681-1. 14, 15, 16, 17, 18

GUO, J. et al. RepoAudit: An autonomous LLM-agent for repository-level code auditing. In: *Proceedings of the 42nd International Conference on Machine Learning*. [s.n.], 2025. Disponível em: <<https://openreview.net/pdf?id=TXcifVbFpG>>. 28

HEVNER, A. R. et al. Design science in information systems research. *MIS Quarterly*, JSTOR, v. 28, n. 1, p. 75–105, 2004. Disponível em: <https://wise.vub.ac.be/sites/default/files/thesis_info/design_science.pdf>. 32

HOU, Z. et al. Generation, migration and optimization of cross-language static-analysis rules based on large language models. In: *Proceedings of the 2025 IEEE/ACM International Conference on Software Engineering (ICSE)*. [s.n.], 2025. Disponível em: <<https://ieeexplore.ieee.org/document/11173453/>>. 3, 5, 6, 7, 8, 9, 20, 25, 26, 27, 32

JIANG, Z. et al. Fact-aligned and template-constrained static analyzer rule enhancement with llms. In: *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. [S.l.: s.n.], 2025. 5, 6, 20, 21

JURAFSKY, D.; MARTIN, J. H. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*. 3rd draft. ed. [s.n.], 2026. Em preparação. Disponível em: <https://web.stanford.edu/~jurafsky/slp3/ed3book_jan26.pdf>. 19

LI, F. et al. *LLM-based Vulnerability Detection at Project Scale: An Empirical Study*. 2026. Disponível em: <<https://arxiv.org/pdf/2601.19239>>. 4, 9, 18, 24, 27, 28, 29, 30

MØLLER, A.; SCHWARTZBACH, M. I. *Static Program Analysis*. Department of Computer Science, Aarhus University, 2024. Notas de aula. Disponível em: <<https://cs.au.dk/~amoeller/spa/>>. Acesso em: 27 abr. 2026. 5

NSA Center for Assured Software. *Juliet C/C++ 1.3 - NIST Software Assurance Reference Dataset (SARD) Test Suite #112*. [S.l.]: National Institute of Standards and Technology (NIST), 2017. <<https://samate.nist.gov/SARD/test-suites/112>>. Acesso em: 29 maio 2026. 24, 32, 33

SEACORD, R. C. *Secure Coding in C and C++*. 2nd. ed. Addison-Wesley Professional, 2013. ISBN 978-0321822130. Disponível em: <<https://raw.githubusercontent.com/DarkCodeOrg/welcome-to-cybersecurity/master/securecodingincandc++.pdf>>. Acesso em: 27 abr. 2026. 18

Software Engineering Institute. *SEI CERT C Coding Standard: Rules for Developing Safe, Reliable, and Secure Systems*. 2016 edition. ed. Carnegie Mellon University, 2016. Acessado em: 3 de junho de 2026. Disponível em: <https://chenweixiang.github.io/docs/SEI_CERT_C_Coding_Standard_2016_Edition.pdf>. 18

TANG, Y. et al. Grammar-based patches generation for automated program repair. In: *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*. Association for Computational Linguistics, 2021. p. 1300–1305. Disponível em: <https://aclanthology.org/2021.findings-acl.111/>. 5, 6

TUNSTALL, L.; WERRA, L. von; WOLF, T. *Natural Language Processing with Transformers: Building Language Applications with Hugging Face*. Sebastopol, CA: O'Reilly Media, Inc., 2022. ISBN 978-1098103248. 19, 20

YANG, C. et al. Knighter: Transforming static analysis with llm-synthesized checkers. *arXiv preprint arXiv:2503.09002*, 2025. Disponível em: <https://arxiv.org/abs/2503.09002>. 3, 6, 7, 8, 22, 23, 24, 26, 28, 32